Theory Assignment - 3

JavaScript

Kanishk saxena

Q1. What is the difference between frontend and backend JavaScript?

Frontend JavaScript runs in the user's web browser (client side) and controls what the user sees and interacts with. Backend JavaScript runs on a server (commonly using Node.js) and handles data, logic, and communication with databases.

Point	Frontend JavaScript	Backend JavaScript
Runs on	Web browser (client)	Server (Node.js)
Purpose	User interface, interactivity, animations, form validation	Business logic, database access, authentication, APIs
Access to	DOM, window, events, localStorage	File system, network, databases, OS
Examples / tools	React, Angular, Vue, jQuery	Node.js, Express.js, NestJS
Visibility	Code is visible to the user	Code is hidden on the server

Q2. What is DOM?

DOM stands for Document Object Model. It is a programming interface created by the browser when a web page loads. The DOM represents the HTML document as a tree of objects (nodes), where every element, attribute, and piece of text is a node. JavaScript uses the DOM to read, change, add, or delete HTML content, attributes, and styles dynamically.

Example:

```
let heading = document.getElementById("title");
heading.textContent = "Hello, DOM!";
heading.style.color = "blue";
```

Q3. What is event in JavaScript?

An event is an action or occurrence that happens in the browser, such as a mouse click, key press, form submission, page load, or mouse hover. JavaScript can listen for these events and run a function (event handler) in response.

Common events: click, dblclick, mouseover, keydown, keyup, submit, change, load.

```
const btn = document.getElementById("myBtn");
btn.addEventListener("click", function () {
```

```
  alert("Button clicked!");
});
```

Q4. What is event loop in JavaScript?

JavaScript is single-threaded, meaning it executes one task at a time. The event loop is the mechanism that lets JavaScript handle asynchronous operations (like setTimeout, fetch, and events) without blocking the main thread.

It works with these parts:

- **Call Stack:** where functions are executed one by one.
- **Web APIs:** browser features (timers, DOM events, fetch) that handle async tasks in the background.
- **Callback (Task) Queue:** holds callbacks from timers and events waiting to run.
- **Microtask Queue:** holds promise callbacks; it has higher priority than the callback queue.
- **Event Loop:** continuously checks if the call stack is empty and, if so, moves the next task from the queues to the stack.

Example:

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("End");

// Output: Start, End, Promise, Timeout
```

Q5. What is hoisting in JavaScript?

Hoisting is JavaScript's default behavior of moving declarations to the top of their scope (before code execution). This means variables and functions can be used before they appear in the code.

- **var:** hoisted and initialized with undefined.
- **let and const:** hoisted but not initialized (Temporal Dead Zone), so using them early causes a ReferenceError.
- **Function declarations:** fully hoisted, so they can be called before their definition.

```
console.log(a);   // undefined
var a = 10;

sayHi();          // works
function sayHi() { console.log("Hi"); }
```

```
console.log(b);   // ReferenceError
let b = 5;
```

Q6. What is event bubbling?

Event bubbling is a process in which an event triggered on an inner (child) element propagates upward to its parent elements, then to the grandparent, and so on up to the document. So if you click a button inside a div, the button's click handler runs first, then the div's handler, then the body's handler.

```
<div id="parent">
  <button id="child">Click</button>
</div>

parent.addEventListener("click", () => console.log("Parent"));
child.addEventListener("click", () => console.log("Child"));

// Clicking the button prints: Child, then Parent
```

Bubbling can be stopped using event.stopPropagation(). The opposite process, where the event travels from the top down to the target, is called event capturing.

Q7. What is the difference between named function, anonymous function and arrow function?

- **Named function:** a function that has a name. It is easy to reuse and appears with its name in debugging.

```
function add(a, b) {
 return a + b;
}
```

- **Anonymous function:** a function without a name, usually assigned to a variable or passed as an argument.

```
const add = function (a, b) {
 return a + b;
};
```

- **Arrow function:** a shorter syntax introduced in ES6 using =>. It does not have its own this, arguments, or prototype, and cannot be used as a constructor.

```
const add = (a, b) => a + b;
```

Feature	Named	Anonymous	Arrow
Has a name	Yes	No	No (assigned to variable)
Hoisted	Yes (declaration)	No	No

Own 'this'	Yes	Yes	No (uses outer this)
Syntax	function name(){}	function(){}	() => {}
Use as constructor	Yes	Yes	No

Q8. Explain for-of and for-in loop?

for-in loop: iterates over the enumerable property names (keys) of an object. With arrays it gives the index as a string.

```
const person = { name: "Ali", age: 20 };
for (let key in person) {
  console.log(key, person[key]);   // name Ali, age 20
}
```

for-of loop: iterates over the values of iterable objects such as arrays, strings, Maps, and Sets. It does not work directly on plain objects.

```
const colors = ["red", "green", "blue"];
for (let color of colors) {
  console.log(color);   // red, green, blue
}
```

Point	for-in	for-of
Iterates over	Keys / property names	Values
Best used with	Objects	Arrays, strings, Maps, Sets
Works on plain objects	Yes	No

Q9. Explain any five array methods?

1. push() - Adds one or more elements to the end of an array and returns the new length.

```
let arr = [1, 2, 3];
arr.push(4);   // [1, 2, 3, 4]
```

2. pop() - Removes the last element of an array and returns it.

```
let arr = [1, 2, 3];
arr.pop();   // returns 3, arr = [1, 2]
```

3. map() - Creates a new array by applying a function to every element.

```
let nums = [1, 2, 3];
let dbl = nums.map(n => n * 2);   // [2, 4, 6]
```

4. filter() - Creates a new array with only the elements that pass a given condition.

```
let nums = [1, 2, 3, 4];
let even = nums.filter(n => n % 2 === 0);   // [2, 4]
```

5. reduce() - Reduces the array to a single value by applying a function with an accumulator.

```
let nums = [1, 2, 3, 4];
let sum = nums.reduce((acc, n) => acc + n, 0);   // 10
```

Other useful methods: shift(), unshift(), slice(), splice(), forEach(), find(), includes(), sort().

Q10. What is callback method?

A callback is a function that is passed as an argument to another function and is executed later, after the other function has finished its work or when an event occurs. Callbacks are widely used for asynchronous tasks such as timers, event handling, and reading files.

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

function done() {
  console.log("Greeting finished");
}

greet("Sara", done);
// Output: Hello Sara, Greeting finished
```

Example with setTimeout:

```
setTimeout(function () {
  console.log("Runs after 2 seconds");
}, 2000);
```

Note: Too many nested callbacks cause "callback hell", which is avoided using Promises and async/await.